EE627 — Class Summary

April 10, 2026

Prof. Rensheng Wang

Contents

1 Recap and Course Philosophy **2**
1.1 Why Revisit Recommendation . . . 2
1.2 Why Recommendation Matters in Industry . . . 2

2 Two Families of Recommender Systems **2**
2.1 Content-Based Filtering . . . 2
2.2 Collaborative Filtering . . . 3

3 Neighborhood Methods **3**
3.1 Core Idea . . . 3
3.2 Real-World Analogy . . . 3
3.3 Procedure for the Yahoo Music Project . . . 3

4 Latent Factor Methods **3**
4.1 Core Idea . . . 3
4.2 What "Latent" Means . . . 4
4.3 Dot-Product Rating Rule . . . 4
4.4 Geometric Picture . . . 4

5 From SVD to Matrix Factorization **4**
5.1 Motivation: Rank-K Approximation . . . 4
5.2 Compression Example . . . 5
5.3 Explicit vs. Implicit Feedback . . . 5
5.4 Ratings Matrices Are Overwhelmingly Sparse . . . 5

6 Matrix Factorization Algorithm (Homework 7) **6**
6.1 Problem Statement . . . 6
6.2 Regularized Loss Function . . . 6
6.3 Stochastic Gradient Descent Updates . . . 6
6.4 Numerical Walk-Through from Lecture . . . 6
6.5 Important Implementation Rule . . . 6

7 Alternative Learning Algorithm: Alternating Least Squares **7**
7.1 When to Prefer ALS Over SGD . . . 7

8 Adding Biases **7**
8.1 Why Plain Dot Products Are Not Enough 7
8.2 Bias-Augmented Prediction 7
8.3 Full Regularized Loss with Biases 7

9 Homework 7 Deliverable **8**
9.1 Test Case 1 — 5×4 Matrix, $K = 2$ 8
9.2 Test Case 2 — 6×5 Matrix, $K = 3$ 8

10 Key Formulas Reference **8**

1 Recap and Course Philosophy

1.1 Why Revisit Recommendation

The Yahoo Music recommendation task (the same dataset used in Homework 4 and the midterm) will be revisited repeatedly, each time with a *different target method*. The point is not the dataset but the practice of attacking the same business problem from multiple angles:

- Homework 4 used very simple **heuristic scoring** (album / artist / genre aggregation).
- The midterm extended that heuristic with feature engineering and cold-start fallbacks.
- Homework 7 (this week) introduces **matrix factorization** as a principled alternative.
- Subsequent weeks will replace the single-machine MATLAB implementation with **PySpark** so the same logic runs at scale.

1.2 Why Recommendation Matters in Industry

The professor reiterated that recommendation systems are the most immediately profitable application of machine learning:

- They convert one dollar of compute into roughly ten dollars of revenue — a ratio that no physical-goods business can match.
- They monetize *instantly*: the recommendation is served, the click happens, and the revenue arrives within seconds. Unlike manufacturing, there is no months-long design-to-cash cycle.
- Interviews at major tech and finance companies therefore stress *design-level* recommender questions ("Given this data and this business goal, how would you approach it?") rather than code-writing drills.

2 Two Families of Recommender Systems

2.1 Content-Based Filtering

Idea: Describe each item with an explicit attribute vector and recommend items whose attributes match the user's stated or inferred preferences.

Example attributes (movies):

- Genre: war, documentary, crime, fantasy, romance, ...
- Cast and director.
- Box-office popularity.
- Release year, runtime, language, rating, ...

Limitation: Content-based filtering requires someone to *manually curate* every item's attributes. This works for studio releases but fails for user-generated content (e.g., a YouTube upload) where no one has filled in the metadata.

2.2 Collaborative Filtering

Idea: Ignore item attributes entirely and learn *only* from the user–item interaction matrix.

- Fully data-driven — no manual curation required.
- Works uniformly across domains (movies, music tracks, shopping items, web pages, social posts).
- Is the **mainstream** approach for industrial recommenders today.

Collaborative filtering itself splits into two sub-families, discussed next:

1. **Neighborhood methods** (Section 3)
2. **Latent factor methods** (Section 4)

3 Neighborhood Methods

3.1 Core Idea

A user's "neighborhood" is *not* geographic. It is the set of other users whose **historical ratings overlap** the target user's history. These users are treated as surrogate friends whose opinions can be aggregated.

3.2 Real-World Analogy

If Joe wants a movie recommendation, he asks his buddies what they watched and takes whichever title received the most votes. Neighborhood collaborative filtering automates this: the algorithm picks the buddies for Joe by looking at shared rating history.

3.3 Procedure for the Yahoo Music Project

Given a target (test) user Joe with 6 tracks to score and a training history of 10 tracks:

1. For every other user u, count how many of Joe's 10 training tracks also appear in u's history.
2. Rank users by overlap. The highest-overlap users (e.g., one with 9/10 matches and another with 5/10) become Joe's **neighborhood**.
3. For each of Joe's 6 test tracks, collect ratings of that track from the neighborhood users.
4. Average those ratings (weighting by overlap if desired) to produce Joe's predicted rating.

Interview takeaway: This is the type of question asked in first-round technical screens. The interviewer does not want code — they want a clear verbal explanation of how the neighborhood is formed and how its votes are aggregated into a prediction.

4 Latent Factor Methods

4.1 Core Idea

Rather than finding similar users, project *every* user *and every* item into a common K-dimensional **latent space**. A user who sits near an item in that space is predicted to rate it highly.

4.2 What "Latent" Means

Each coordinate in the latent space is a *factor* — a number that summarizes one aspect of a user or item. In a realistic deployment the rank is typically $f = 20$ to 100 factors. Some of these may correspond to obvious dimensions (comedy vs. drama, amount of action, orientation to children); others may be subtler (depth of character development, quirkiness); others are fully uninterpretable. Example interpretable factors from the lecture's 2-D illustration:

- Gender orientation (male $\leftrightarrow$ female audience).
- Tone (serious $\leftrightarrow$ escapist).

In practice most factors have **no explicit meaning** the algorithm can name; the model just learns whatever directions best explain the observed ratings. Hence the word "latent." For a user u, each factor coordinate measures how much u likes items that score high on the corresponding item factor, and the user–item dot product measures their overall compatibility.

4.3 Dot-Product Rating Rule

For user u with latent vector $\mathbf{p}_u \in \mathbb{R}^K$ and item i with latent vector $\mathbf{q}_i \in \mathbb{R}^K$, the predicted rating is

$$\hat{r}_{ui} \;=\; \mathbf{p}_u^\top \mathbf{q}_i \;=\; \sum_{k=1}^{K} p_{uk}\, q_{ik}. \tag{1}$$

Intuition via a 2-D toy example:

	$\mathbf{p}_u$	$\mathbf{q}_i$
Aligned	$(1,\ -1)$	$(1,\ -1)$
Opposite	$(1,\ -1)$	$(-1,\ \ 1)$
Orthogonal	$(1,\ -1)$	$(1,\ \ 1)$

The aligned pair yields $\mathbf{p}_u^\top \mathbf{q}_i = 2$ (strong recommendation); the opposite pair yields -2 (anti-recommendation); the orthogonal pair yields 0 (neutral).

4.4 Geometric Picture

In a 2-D latent space, both users and items are plotted as points. To recommend for user *Dave*, locate Dave's point and return the items nearest to him (largest dot product, i.e., shortest angular distance). A user near *The Lion King* will be recommended *The Lion King*, not *The Princess Diaries*, because the first item's latent vector aligns with the user's and the second does not.

5 From SVD to Matrix Factorization

5.1 Motivation: Rank-K Approximation

A full ratings matrix $R \in \mathbb{R}^{M \times N}$ with M = users and N = items is enormous — at industrial scale, $M \approx 10^9$ and $N \approx 10^9$. Storing or transmitting R directly is infeasible.

The SVD insight from 1990s image compression: any $M \times N$ matrix can be approximated by the product of two much smaller matrices,

$$R_{M\times N} \;\approx\; P_{M\times K}\, Q_{K\times N}, \tag{2}$$

where $K \ll \min(M, N)$. The approximation has rank K, and increasing K yields better fidelity at the cost of more storage.

5.2 Compression Example

For an $M = N = 1000$ image:

- Full storage: $M \cdot N = 10^6$ entries.
- Rank-2 approximation: $2M + 2N = 4000$ entries.
- Compression ratio: $\approx 250\times$ smaller.

5.3 Explicit vs. Implicit Feedback

Matrix factorization is flexible about the kind of interaction data it ingests:

- **Explicit feedback** — users directly state preferences (Netflix star ratings, TiVo thumbs up/down). The resulting R is **sparse** because any single user rates only a tiny slice of the catalog.
- **Implicit feedback** — preferences are inferred from behavior (purchase history, browsing, search patterns, clicks, even mouse movements). Implicit matrices are usually **dense** because "no action" is itself a (weak) signal.

This homework and the Yahoo Music project use explicit ratings, so the focus is the sparse case.

5.4 Ratings Matrices Are Overwhelmingly Sparse

Unlike an image, the explicit-feedback ratings matrix R is almost entirely **empty** — a typical user has rated only a few dozen items out of millions. Zero entries in R represent "no observation," *not* "rated zero."

Why conventional SVD fails here: SVD is undefined for matrices with missing entries. Earlier systems tried to *impute* the missing values to make R dense, but imputation is expensive and inaccurate imputation distorts the factors. The modern answer is to **model only the observed entries directly** and control overfitting with regularization.

Consequence for matrix factorization:

- Only *observed* (nonzero) entries enter the loss function.
- The learned P and Q can then be multiplied to supply predictions for the missing entries — which is exactly the recommendation output.

6 Matrix Factorization Algorithm (Homework 7)

6.1 Problem Statement

Given the observed ratings $R \in \mathbb{R}^{M \times N}$ and a chosen rank K, find

$$P \in \mathbb{R}^{M \times K}, \qquad Q \in \mathbb{R}^{K \times N}$$

such that PQ closely matches R on the observed entries while remaining smooth elsewhere.

6.2 Regularized Loss Function

Let $\mathbf{p}_u$ denote row u of P and $\mathbf{q}_i$ denote column i of Q. The loss summed over observed entries is

$$\mathcal{L} \;=\; \sum_{(u,i)\,\in\,\mathrm{obs}} \Big(r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i\Big)^2 \;+\; \frac{\beta}{2}\Big(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2\Big). \tag{3}$$

The first term penalizes reconstruction error; the second (regularization) term penalizes large factor magnitudes so that the model cannot overfit the observed entries.

6.3 Stochastic Gradient Descent Updates

For each observed entry, compute the error

$$e_{ui} \;=\; r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i \tag{4}$$

and apply the coupled updates

$$p_{uk} \leftarrow p_{uk} + \alpha\,\big(2\,e_{ui}\,q_{ik} - \beta\,p_{uk}\big), \tag{5}$$

$$q_{ik} \leftarrow q_{ik} + \alpha\,\big(2\,e_{ui}\,p_{uk} - \beta\,q_{ik}\big), \tag{6}$$

where α is the learning rate and β is the regularization strength.

6.4 Numerical Walk-Through from Lecture

Suppose $K = 2$, $\alpha = 0.01$, and we observe $r_{11} = 100$. With random initialization

$$\mathbf{p}_1 = (10,\ 9), \qquad \mathbf{q}_1 = (7,\ 8),$$

the prediction is $10 \cdot 7 + 9 \cdot 8 = 142$, so the error is $e_{11} = 100 - 142 = -42$. The update for $\mathbf{q}_1$ is

$$\mathbf{q}_1 \;\leftarrow\; \mathbf{q}_1 + \alpha \cdot 2e_{11} \cdot \mathbf{p}_1,$$

which nudges $(7, 8)$ in the direction that reduces the reconstruction error. The matching update runs simultaneously for $\mathbf{p}_1$. Repeating for every observed entry constitutes one epoch; the algorithm loops for `steps` epochs (or until the loss drops below a threshold).

6.5 Important Implementation Rule

Zero entries are skipped in both the loss and the updates. In MATLAB a rating of 0 is used as a sentinel for "missing." Failing to skip zeros will bias the factors toward predicting low ratings everywhere.

7 Alternative Learning Algorithm: Alternating Least Squares

Equation (1) is *not convex* in P and Q jointly, but if one of the two matrices is held fixed the remaining subproblem is a standard quadratic least-squares problem. **Alternating Least Squares (ALS)** exploits this:

1. Fix P; solve for each column of Q independently as a ridge regression.
2. Fix Q; solve for each row of P independently as a ridge regression.
3. Repeat until convergence. Each step is guaranteed to decrease the loss.

7.1 When to Prefer ALS Over SGD

SGD is generally simpler and faster, but ALS wins in two cases:

- **Parallelization.** Because each $\mathbf{q}_i$ (resp. $\mathbf{p}_u$) update is independent of the other items (resp. users), ALS parallelizes trivially across many machines. This is exactly why **PySpark's** `mllib.recommendation.ALS` — the algorithm that will drive next week's lecture — uses ALS rather than SGD.
- **Implicit-feedback data.** Implicit matrices are dense, so SGD would have to loop over *every* cell; ALS solves each user or item slice in closed form and handles the density efficiently.

8 Adding Biases

8.1 Why Plain Dot Products Are Not Enough

Much of the variation in observed ratings has nothing to do with user–item compatibility. Some users rate everything generously; others are critical. Some items are universally loved; others are universally disliked. These are **bias** effects, and attributing them to the latent factors wastes representational capacity.

8.2 Bias-Augmented Prediction

Introduce a global mean μ, a per-user bias b_u, and a per-item bias b_i. The predicted rating becomes

$$\hat{r}_{ui} \;=\; \mu \;+\; b_u \;+\; b_i \;+\; \mathbf{p}_u^\top \mathbf{q}_i. \tag{7}$$

Example from the slides. Suppose the global average movie rating is $\mu = 3.7$ stars. *Titanic* is half a star above average ($b_i = +0.5$). Joe is a critical user who rates 0.3 stars below average ($b_u = -0.3$). A first-order estimate of Joe's rating for *Titanic* is

$$3.7 + 0.5 - 0.3 \;=\; 3.9 \text{ stars.}$$

8.3 Full Regularized Loss with Biases

The biased model is learned by minimizing

$$\sum_{(u,i)\,\in\,\text{obs}} \left(r_{ui} - \mu - b_u - b_i - \mathbf{p}_u^\top \mathbf{q}_i\right)^2 \;+\; \lambda\left(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2 + b_u^2 + b_i^2\right). \tag{8}$$

The biases typically absorb a large share of the observed signal, so modelling them carefully is more important than adding more latent factors. The plain dot-product model in Homework 7 is a simplification — it omits the bias terms so the algebra stays transparent.

9 Homework 7 Deliverable

The assignment is to implement a MATLAB function

```
[nP, nQ] = matrix_factorization(R, P, Q, K, steps, alpha, beta)
```

that realizes the SGD procedure above, and to verify it on the sample rating matrices provided in `testMatrixFactor.m`.

9.1 Test Case 1 — 5×4 Matrix, $K = 2$

The primary test mirrors the instructions: a 5×4 rating matrix, rank $K = 2$, `steps = 5000`, $\alpha = 0.0002$, $\beta = 0.02$. A correct implementation reconstructs the observed entries to within a fraction of a rating unit (the slide deck shows a reference $\hat{R}$ with entries 5.0272, 2.8334, ..., 4.0341, which agrees to about two decimals with the observed entries of R).

9.2 Test Case 2 — 6×5 Matrix, $K = 3$

The same script runs a second, larger stress test: a 6×5 matrix with rank $K = 3$ and `steps = 10000` (same α and β). The larger rank and longer run are needed because the higher-dimensional matrix has more structure to absorb. Passing both cases demonstrates that the implementation is generic and not tuned to a single input.

Next week the same factorization logic will be re-implemented in **PySpark** — specifically `pyspark.ml.recommenda` — and used as the core algorithm for an updated Yahoo Music recommendation submission. The switch from SGD to ALS is driven by the parallelization argument in Section 7.

10 Key Formulas Reference

Predicted rating (plain): $\hat{r}_{ui} = \mathbf{p}_u^\top \mathbf{q}_i = \sum_{k=1}^{K} p_{uk}\, q_{ik}$

Predicted rating (biased): $\hat{r}_{ui} = \mu + b_u + b_i + \mathbf{p}_u^\top \mathbf{q}_i$

Observed error: $e_{ui} = r_{ui} - \mathbf{p}_u^\top \mathbf{q}_i$

Regularized loss: $\mathcal{L} = \sum_{(u,i)\in\text{obs}} e_{ui}^2 + \frac{\beta}{2}\big(\|\mathbf{p}_u\|^2 + \|\mathbf{q}_i\|^2\big)$

SGD update (user): $p_{uk} \leftarrow p_{uk} + \alpha\big(2e_{ui}q_{ik} - \beta p_{uk}\big)$

SGD update (item): $q_{ik} \leftarrow q_{ik} + \alpha\big(2e_{ui}p_{uk} - \beta q_{ik}\big)$

Rank-K approximation: $R_{M\times N} \approx P_{M\times K}\, Q_{K\times N}, \quad K \ll \min(M, N)$